

This document demonstrates the use of GitHub Copilot CLI
Find the screenshot of the interaction below:

```
> I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message

● The user wants to combine their last two commits into one with a new message. This is a git rebase operation. I should show them how to do it.

1. First check the git status and log to see the current state
2. Use interactive rebase to squash the last two commits
3. Allow them to write a new commit message

The command would be `git rebase -i HEAD~2` to interactively rebase the last 2 commits, then mark the second one as "squash" or "s" to combine it with the first.

● I'll help you combine the last two commits. Let me first check your git status and recent commits.

● View last 3 commits
$ cd /home/mbelenga/Desktop/set-se-103/advanced-shell-practice && git --no-pager log --oneline -3
└─ 4 lines...

● I can see your last two commits. I'll use an interactive rebase to combine them. This will open an editor where you write your new commit message.

● Squash last two commits
$ cd /home/mbelenga/Desktop/set-se-103/advanced-shell-practice && GIT_EDITOR="sed -i '2s/^pick/squash/'" git rebase HEAD~2
└─ 14 lines...

● It looks like the commits might have similar or no changes. Let me check the actual differences and use a reset approach instead:

● Abort rebase, soft reset 2 commits, check status
$ cd /home/mbelenga/Desktop/set-se-103/advanced-shell-practice && git rebase --abort && git reset --soft HEAD~2 && git status
└─ 6 lines...
```

GitHub Copilot CLI suggested using the following command:

git rebase -i HEAD~2

This command allows interactive rebasing, where the last two commits can be squashed into a single commit and replaced with a new commit message.

Accuracy Ranking and Justification

Accuracy Rating: 8 / 10

Justification:

The AI's response accurately addresses the problem by recommending an interactive rebase, which is the correct and safest approach for combining local commits that have not yet been pushed.

Conclusion

GitHub Copilot CLI effectively translated a natural language Git problem into a precise and correct command. This demonstrates how AI-assisted tools can improve developer productivity and reduce errors when working with complex shell and Git operations. I have learnt something really cool today while tackling this project